

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 489: Mobile Application Development

Lab Exam — Smart Geo-Tagged Landmarks (v5)

📌 Objective

Design and develop an Android mobile application that interacts with a faculty-provided REST API to manage and visualize Smart Geo-Tagged Landmarks.

Each landmark includes:

- Title
- Geographic location (latitude & longitude)
- Image
- Activity-based score (computed by server)

Your application must support viewing, visiting, filtering, and offline handling of landmarks.

⚠️ Important Rules

- You are **NOT** allowed to build or modify the backend
- You must use the provided API only
- Each student will receive a unique API key, valid for this semester only
- API behavior (e.g., scoring) may vary per student

🔗 API Details

You will be provided with:

`https://labs.anontech.info/cse489/exm3/api.php`

You must include your ID in every request as `key`:

`?action=get_landmarks&key=YOUR_KEY`

If your key is invalid, every endpoint will respond with:

```
HTTP 403
{"error": "invalid_or_expired_key"}
```

View submission online: `https://labs.anontech.info/cse489/exm3`

📡 Available API Endpoints

1. Get Landmarks

`GET ?action=get_landmarks&key=YOUR_KEY`

Returns a list of landmarks with: `id`, `title`, `lat`, `lon`, `image`, `score`, `visit_count`, `avg_distance`.

2. Visit Landmark (now asynchronous — see below)

`POST ?action=visit_landmark&key=YOUR_KEY`

Body (JSON):

```
{
  "landmark_id": 1,
  "user_lat": 23.7,
  "user_lon": 90.4
}
```

Response (immediate — the visit is **not** processed yet):

```
{ "job_id": 42, "status": "pending" }
```

3. Get Job Status (new)

`GET ?action=get_job_status&key=YOUR_KEY&job_id=42`

Poll this until `status` is `done`:

```
{ "job_id": 42, "status": "pending" }
```

```
{ "job_id": 42, "status": "done", "distance": 123.45 }
```

A `job_id` belongs to the key that created it — you cannot query another student's job. An unknown `job_id` returns `404 {"error":"job_not_found"}`.

The server processes jobs in the background; expect a short delay (a few seconds) before a job moves from `pending` to `done`. Your app must poll, not assume the result is ready instantly.

4. Create Landmark

`POST ?action=create_landmark&key=YOUR_KEY`

5. Delete Landmark (Soft Delete)

`POST ?action=delete_landmark&key=YOUR_KEY`

6. Restore Landmark

`POST ?action=restore_landmark&key=YOUR_KEY`

🧠 Core Functional Requirements

1. Landmarks Display

- Fetch landmarks from API
- Display: Title, Image, Score
- Handle dynamic data correctly

2. Map View

- Show all landmarks on map
- Center map on Bangladesh
- Marker must:
  - Represent a landmark
  - Show details on click
  - Marker color should reflect score (low → high)

3. Visit Feature (IMPORTANT)

- User can visit a landmark
- App must:
  - Get current GPS location
  - Send visit request (`visit_landmark`) and receive a `job_id`
  - **Poll `get_job_status` in the background until the job is `done`** — do not block the UI thread and do not assume the distance is available in the `visit_landmark` response itself
  - Display the returned distance once the job completes
  - Show success/failure message

4. Landmarks List

- Show all landmarks in list form
- Each item must show: Title, Score, Image
- Must support: Sorting by score, Filtering by minimum score

5. Activity Screen (Visit History)

- Display recent visits
- Each entry must show: Landmark name, Visit time, Distance

6. Add Landmark

- Input: Title, Latitude & Longitude, Image
- Auto-fetch GPS location for new entry

Most Common mistakes: if you send `raw JSON`, `$_FILES` will be empty server-side. For file upload you **must** use `Body → form-data`.

7. Soft Delete Handling

- Deleted landmarks should not appear in list
- App must not crash if data changes

8. Offline Support (MANDATORY)

Your app must:

- Cache fetched data locally
- Display data when offline
- Queue visit requests when offline
- Sync queued requests when internet is available

9. Error Handling

- Show success messages using Toast/Snackbar
- Show errors using dialogs or messages
- Handle API failures gracefully

10. Background Job Queue

The backend now processes visits asynchronously (see API section above). Your app is required to use **WorkManager** (or an equivalent constrained/guaranteed background-work API — not a manual `Thread`/timer loop) to:

1. **Poll `get_job_status`** for any pending visit job(s) until they resolve, then update your local cache/UI with the result.
2. **Drain the offline visit queue** from Requirement 8 once connectivity returns, with retry/backoff on failure.

These are the same underlying problem (reliable background work that must survive app restarts and unreliable connectivity) — one WorkManager-based mechanism should reasonably serve both.

📱 UI Requirements

Use Bottom Navigation with 4 tabs: Map, Landmarks, Activity, Add/View.

📦 Submission Instructions

🔗 GitHub Repository

- Create a private repository
- Add instructor as collaborator (GitHub handle: `rahman9909`)
- Submit repository link via form

📁 Required Structure

```
project-root/
├── app/ (contains your application codebase without Build folder)
├── Ai_usage.txt (your justification/spec of ai usage)
└── README.txt
```

📝 README.txt Must Include

- Project Overview
- Features Implemented
- API Usage
- Offline Strategy
- Architecture Used
- Challenges Faced

📊 Evaluation Criteria

AI usage will be evaluated by your AI and git usage with code analysis.

🚫 Academic Integrity

- Do NOT share complete solutions
- Code similarity will be checked

You may: discuss ideas, share small snippets.

💡 Tips

Start with API integration first. Then implement UI. Then add offline support. Test frequently.

⏰ Deadline

Strict deadline by 15th August 2026 midnight. (no late submission accepted)

Submission via form: https://forms.gle/3YvyKYNwb824yeH17

Only ONE submission allowed.

✅ Final Note

This assignment tests your ability to:

- Work with real APIs
- Handle dynamic data
- Manage offline behavior and background work
- Build complete mobile applications

Additional Resources

1. Google Maps Android API
   https://developers.google.com/maps/documentation/android-sdk/overview
2. OpenStreetMap Integration in Android
   https://wiki.openstreetmap.org/wiki/Android
3. Retrofit, A Type-Safe HTTP Client for Android
   https://square.github.io/retrofit/
4. Save Data in a Local Database Using Room
   https://developer.android.com/training/data-storage/room
5. CameraX
   https://developer.android.com/media/camera/camerax
6. Photo Picker
   https://developer.android.com/training/data-storage/shared/photo-picker
7. Get the Last Known Location
   https://developer.android.com/develop/sensors-and-location/location/retrieve-current
8. WorkManager — background work
   https://developer.android.com/topic/libraries/architecture/workmanager

Tip (not graded): a Repository pattern / single-source-of-truth architecture (Room as the source of truth, background workers write to it, UI observes it) pairs naturally with the offline caching and background queue requirements above.

Good luck with your exam!